

Googol

Motor de Busca Distribuído

Relatório de Arquitetura do Sistema



Sistemas Distribuídos

Universidade de Coimbra
Departamento de Engenharia Informática

Autores:

Henrique Diz	2023213681
Rodrigo Manão	2023207589
João Francisco	2023228417

13 de dezembro de 2025

Conteúdo

1	Introdução	5
1.1	Visão Geral	5
1.2	Objetivos do Sistema	5
1.3	Tecnologias Utilizadas	5
2	Arquitetura Global do Sistema	6
2.1	Diagrama de Arquitetura	6
2.2	Componentes Principais	6
2.3	Fluxo de Dados	7
2.3.1	Fluxo de Indexação	7
2.3.2	Fluxo de Pesquisa	7
3	Backend - Camada RMI	8
3.1	Gateway	8
3.1.1	Responsabilidades	8
3.1.2	Interface RMI	8
3.1.3	Algoritmo de Balanceamento de Carga	9
3.1.4	Algoritmo de Pesquisa de palavra(s)	9
3.2	Barrels (Servidores de Índice)	10
3.2.1	Estruturas de Dados	10
3.2.2	Replicação de Índice	11
3.3	Deteção Dinâmica de Stopwords	13
3.3.1	Algoritmo de Deteção	13
3.4	URL Queue	13
3.4.1	Gestão de Prioridades	13
3.5	Downloader	14
3.5.1	Pipeline de Processamento	14
3.5.2	Retry Logic	14
3.6	Persistência e Recuperação	14
3.6.1	Backup de Estado	14
3.6.2	Recuperação após Falha	15
4	Backend - WebApp (Spring Boot)	16
4.1	Arquitetura em Camadas	16
4.2	REST API Endpoints	16
4.3	DTOs (Data Transfer Objects)	16
4.3.1	SearchResponseDTO	16
4.3.2	SearchResultDTO	17
4.4	Segurança e SSL	17
4.5	Integração RMI via Spring	17
4.6	Análise Contextual com IA	18
4.7	Hackernews	18
4.7.1	HackerNewsService	18
4.7.2	Fluxo de Indexação	18
4.7.3	Controlo de Execução	19

4.7.4	Identificação de Conteúdo HackerNews	19
4.7.5	Vantagens da Integração	19
5	Frontend - React/Next.js	20
5.1	Stack Tecnológica	20
5.2	Estrutura de Rotas	20
5.3	Componentes Principais	20
5.3.1	AnimatedList	20
5.3.2	Orb (WebGL)	20
5.3.3	Cursor Customizado	21
5.3.4	StaggeredMenu	21
5.3.5	ProfileCard	21
5.4	Sistema de Cores por Rota	22
5.5	Funcionalidades Avançadas	22
5.5.1	Análise Contextual com IA	22
5.5.2	Estatísticas em Tempo Real	22
5.5.3	Paginação Inteligente	22
5.6	Otimizações de Performance	23
5.7	Acessibilidade	23
6	Comunicação Entre Componentes	24
6.1	RMI (Remote Method Invocation)	24
6.1.1	Configuração de Registry	24
6.1.2	Padrão de Callbacks	24
6.2	REST API (HTTPS/JSON)	25
6.2.1	Exemplo de Request/Response	25
6.3	CORS (Cross-Origin Resource Sharing)	25
7	Estatísticas e Monitorização	26
7.1	Métricas Coletadas	26
7.2	Dashboard de Estatísticas	26
7.3	Atualização em Tempo Real	26
7.3.1	SSE no Frontend	26
7.3.2	Callbacks RMI (Backend)	27
8	Páginas da Aplicação	28
8.1	Home (/)	28
8.2	Resultados (/results)	28
8.3	Ligações (/ligacoes e /ligacoes/results)	28
8.4	Indexar (/indexar)	28
8.5	Estatísticas (/statistics)	28
8.6	Autores (/autores)	28
9	Integrações	29
9.1	Spring Boot + RPC/RMI	29
9.2	REST WebServices	29
9.3	SSE e plano de WebSockets	29

9.4	Integração opcional com FastAPI	29
10	Testes de Software	30
10.1	Tabela de testes e resultados	30
10.2	Cobertura de cenários	30
11	Distribuição de Tarefas	31
12	Conclusão	32
12.1	Objetivos Alcançados	32
12.2	Destaques Técnicos	32
12.2.1	Backend	32
12.2.2	Frontend	32
12.3	Pontos Fortes	33
12.4	Possíveis Melhorias Futuras	33
12.5	Considerações Finais	33
A	Configuração do Ambiente	34
A.1	Requisitos	34
A.2	Instalação	34
A.2.1	Backend	34
A.2.2	Frontend	34
A.3	Execução	34
A.3.1	Componentes RMI	34
A.3.2	Frontend	34
B	Estrutura de Ficheiros	35
B.1	Backend	35
B.2	Frontend	35

Resumo

Este relatório descreve a arquitetura completa do Googol, um motor de busca distribuído desenvolvido como projeto da disciplina de Sistemas Distribuídos. O sistema utiliza tecnologias como Java RMI para comunicação entre componentes distribuídos, Spring Boot para a camada de API REST, e React/Next.js para o frontend. A arquitetura implementa conceitos avançados como balanceamento de carga, tolerância a falhas, replicação de dados, detecção dinâmica de stopwords, e análise contextual com inteligência artificial.

1 Introdução

1.1 Visão Geral

O Googol é um motor de busca distribuído que implementa uma arquitetura escalável e tolerante a falhas. O sistema é composto por múltiplos componentes que comunicam através de Java RMI (Remote Method Invocation), coordenados por um Gateway central que gere o balanceamento de carga e a agregação de resultados.

1.2 Objetivos do Sistema

- Indexação distribuída de páginas web
- Pesquisa eficiente com suporte a múltiplas palavras
- Balanceamento de carga entre servidores de índice (Barrels)
- Tolerância a falhas com replicação de dados
- Interface web moderna e responsiva
- Análise contextual com IA
- Estatísticas em tempo real

1.3 Tecnologias Utilizadas

- **Backend:** Java 21, Spring Boot 3.2.0, Java RMI, JSoup 1.18.3, tika-core 2.9.2
- **Frontend:** React 19, Next.js 16, TypeScript, CSS, Framer Motion, GSAP, WebGL (OGL)
- **Comunicação:** RMI Registry, REST API (HTTPS/JSON), SSE
- **IA:** Google Gemini API

2 Arquitetura Global do Sistema

2.1 Diagrama de Arquitetura

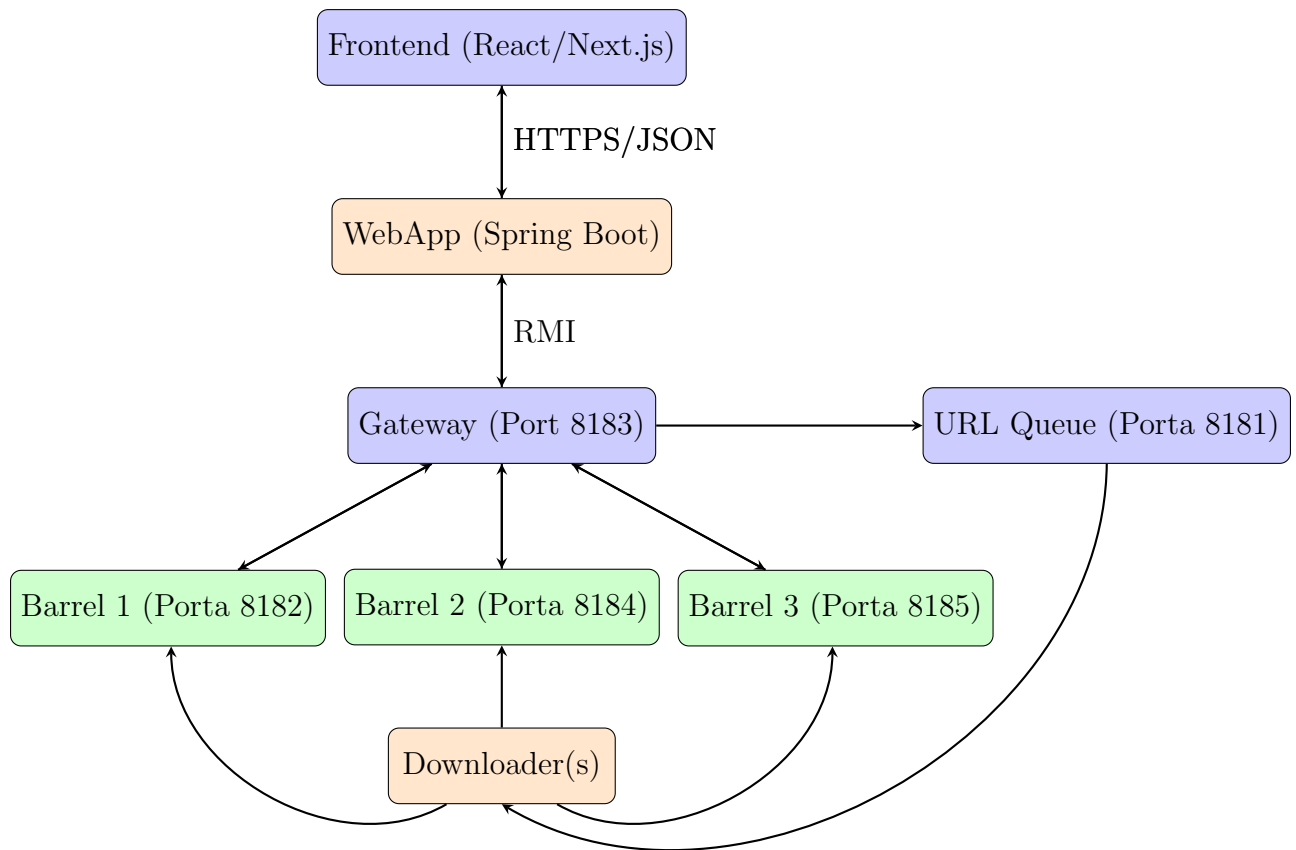


Figura 1: Arquitetura global do sistema Googol

2.2 Componentes Principais

1. **Frontend:** Interface web em React/Next.js
2. **WebApp:** API REST em Spring Boot (HTTPS, porta 8443)
3. **Gateway:** Coordenador central RMI (porta 8183)
4. **Barrels:** Servidores de índice invertido (portas 8182, 8184, 8185)
5. **URL Queue:** Fila de URLs a indexar (porta 8181)
6. **Downloader(s):** Serviços de download e processamento de páginas

2.3 Fluxo de Dados

2.3.1 Fluxo de Indexação

1. Utilizador submete URL através do frontend
2. WebApp recebe pedido e envia para Gateway via RMI
3. Gateway verifica se URL já foi indexado
4. Se não, Gateway adiciona URL à URL Queue
5. Downloader retira URL da fila (operação bloqueante)
6. Downloader faz download da página com JSoup
7. Downloader extrai palavras e links
8. Downloader envia palavras para TODOS os Barrels ativos
9. Cada Barrel armazena no índice invertido e retransmite para outros Barrels
10. Downloader adiciona links descobertos à fila (baixa prioridade)

2.3.2 Fluxo de Pesquisa

1. Utilizador introduz query no frontend
2. Frontend envia pedido GET para WebApp
3. WebApp chama Gateway.searchWords() via RMI
4. Gateway divide query em palavras
5. Para cada palavra, Gateway chama Barrel.searchWord() (round-robin)
6. Gateway faz interseção dos resultados (AND lógico)
7. Gateway obtém contagens de inbound links de todos os Barrels
8. Gateway ordena resultados por número de referências (PageRank-like)
9. Gateway aplica paginação
10. WebApp enriquece resultados com PageInfo (título, descrição)
11. WebApp retorna JSON para frontend
12. Frontend exibe resultados com AnimatedList

3 Backend - Camada RMI

3.1 Gateway

3.1.1 Responsabilidades

O Gateway é o componente central do sistema, responsável por:

- **Balanceamento de carga:** Round-robin entre Barrels ativos
- **Tolerância a falhas:** Retry automático em caso de falha de Barrel
- **Coordenação de pesquisa:** Agregação e interseção de resultados
- **Gestão de Barrels:** Registo, ativação/desativação, sincronização
- **Integração com fila:** Verificação de URLs já indexados
- **Estatísticas:** Coleta de métricas de pesquisa e tempos de resposta
- **Notificações em tempo real:** Callbacks para atualizações de estado

3.1.2 Interface RMI

```
1 public interface GatewayInterface extends Remote {
2     // Pesquisa
3     List<String> searchWordGateway(String word) throws RemoteException;
4     List<String[]> searchWords(List<String> words, int page, int pageSize)
5         throws RemoteException;
6     int getTotalResults(List<String> words) throws RemoteException;
7
8     // Gestao de URLs
9     String addURL(String url, boolean indexAnyway) throws RemoteException;
10    List<String[]> getUrlsForIndexedUrl(String url) throws RemoteException;
11
12    // Gestao de Barrels
13    void registerBarrel(String host, int port, String name)
14        throws RemoteException;
15    void unActiveBarrel(String name, int port) throws RemoteException;
16    List<String> getActiveBarrels() throws RemoteException;
17    List<String> getRegisteredBarrels() throws RemoteException;
18
19    // Estatisticas e Callbacks
20    Map<String, Object> getBarrelStatistics() throws RemoteException;
21    void registerStatsCallback(StatsCallback callback)
22        throws RemoteException;
23    void registerBarrelStateCallback(BarrelStateCallback callback)
24        throws RemoteException;
25 }
```

Listing 1: GatewayInterface - Métodos principais

3.1.3 Algoritmo de Balanceamento de Carga

```

1 private BarrelInterface getNextBarrel() {
2     if (activeBarrels.isEmpty()) return null;
3     BarrelInterface barrel = activeBarrels.get(currentBarrelIndex);
4     currentBarrelIndex = (currentBarrelIndex + 1)%activeBarrels.size();
5     return barrel;
6 }

```

Listing 2: Round-robin load balancing

3.1.4 Algoritmo de Pesquisa de palavra(s)

```

1 public List<String[]> searchWords(List<String> words, int page, int
  pageSize) throws RemoteException {
2
3     // 1) Verificacoes
4     if (words == null || words.isEmpty()) return new ArrayList<>();
5     if (pageSize <= 0) pageSize = 10;
6     if (page < 0) page = 0;
7
8     // 2) Normalizacao da(s) palavra(s) a pesquisar
9     List<String> norm = new ArrayList<>();
10    for (String w : words) {
11        if (w != null && !w.isBlank()) norm.add(w.toLowerCase());
12    }
13    if (norm.isEmpty()) return new ArrayList<>();
14
15    // 3) Resultados devolvidos pela gateway
16    List<HashSet<String>> resultsPerWord = new ArrayList<>();
17    for (String w : norm) {
18        List<String> result = searchWordGateway(w);
19        resultsPerWord.add(new HashSet<>(result));
20    }
21
22    // 4) Devolve a intersecao (apenas quando words.size() > 1)
23    HashSet<String> intersection = new HashSet<>(resultsPerWord.get
  (0));
24    for (int i = 1; i < resultsPerWord.size(); i++) {
25        intersection.retainAll(resultsPerWord.get(i));
26        if (intersection.isEmpty()) break;
27    }
28    if (intersection.isEmpty()) return new ArrayList<>();
29
30    // 5) Atualizamos as estatisticas
31    String searchQuery = String.join(" ", norm);
32    stats.updateSearchStats(searchQuery);
33    notifyStatsUpdateAsync();
34
35    // 6) Numero de referencias de cada url
36    Map<String, Integer> inbound = new HashMap<>();
37    for (BarrelInterface barrel : new ArrayList<>(activeBarrels)) {
38        try {
39            Map<String, Integer> m = barrel.getInboundLinkCounts();
40            if (m != null) {

```

```

41         for (Map.Entry<String, Integer> e : m.entrySet()) {
42             inbound.merge(e.getKey(), e.getValue(), Integer
::sum);
43         }
44     }
45     } catch (RemoteException ignore) {}
46 }
47
48 // 7) Ordenamos
49 List<String[]> ranked = new ArrayList<>();
50 for (String url : intersection) {
51     int refs = inbound.getDefault(url, 0);
52     ranked.add(new String[]{url, String.valueOf(refs)});
53 }
54 ranked.sort((a, b) -> Integer.compare(Integer.parseInt(b[1]),
Integer.parseInt(a[1])));
55
56 // 8) Cortamos para a pagina atual
57 int from = page * pageSize;
58 if (from >= ranked.size())
59     return new ArrayList<>();
60 int to = Math.min(from + pageSize, ranked.size());
61
62 return new ArrayList<>(ranked.subList(from, to));
63 }

```

Listing 3: Interseção de resultados para Pesquisa de palavra(s)

3.2 Barrels (Servidores de Índice)

3.2.1 Estruturas de Dados

```

1 public class IndexStorageBarrel extends UnicastRemoteObject implements
BarrelInterface {
2     // Indice invertido: palavra -> conjunto de URLs
3     private ConcurrentHashMap<String, HashSet<String>> indexedItems;
4
5     // Grafo de URLs: URL -> conjunto de URLs linkados
6     private ConcurrentHashMap<String, HashSet<String>> urlsIndexed;
7
8     // Prevencao de duplicacao
9     private Set<String> receivedMessages;
10
11     // Gestor de stopwords
12     private BarrelStopWords stopWordsManager;
13
14     // Backup da fila
15     private BlockingDeque<String> queueBackup;
16
17     private Set<String> queueSeenUrls;
18 }

```

Listing 4: Estruturas principais dos Barrels

3.2.2 Replicação de Índice

O método *addToIndex* é responsável por adicionar uma palavra e a respetiva URL ao índice local de um barrel. Para garantir consistência e evitar duplicações num ambiente distribuído, cada operação é identificada por um *messageId* único, composto pela palavra e pela URL. Se essa mensagem já tiver sido processada anteriormente, a operação é ignorada.

Antes da indexação, é feita uma verificação para excluir stopwords. Caso a palavra não seja uma stopword, o índice é atualizado, adicionando a URL ao conjunto associado à palavra, criando uma nova entrada se necessário.

Quando o parâmetro *shouldRetransmit* está ativo, a atualização é retransmitida de forma assíncrona para os restantes barrels do sistema. Esta retransmissão é feita fora da secção *synchronized*, recorrendo a um executor, de modo a não bloquear o processamento local.

A função *retransmitToOtherBarrels* obtém, através do gateway, a lista de barrels ativos. Para cada barrel, exceto o próprio, é estabelecida uma ligação RMI e a operação *addToIndex* é invocada remotamente com o parâmetro de retransmissão desativado, evitando ciclos infinitos de propagação. Eventuais falhas de comunicação com outros barrels são registadas, mas não interrompem o funcionamento global do sistema.

Por fim, após cada atualização do índice, o *gateway* é notificado para que possa atualizar o estado dos barrels ativos, permitindo uma gestão consistente da informação no sistema distribuído.

```
1 private void retransmitToOtherBarrels(String word, String url) {
2     try {
3         // Obter lista de barrels do gateway
4         ConfigReader gatewayConfig = new ConfigReader("gateway");
5         Registry gatewayRegistry = LocateRegistry.getRegistry(
6             gatewayConfig.getHost(),
7             gatewayConfig.getPort()
8         );
9         GatewayInterface gateway = (GatewayInterface) gatewayRegistry.
lookup(gatewayConfig.getName());
10        List<String> activeBarrels = gateway.getActiveBarrels();
11
12        for (String barrelInfo : activeBarrels) {
13            String[] parts = barrelInfo.split(":");
14            String barrelName = parts[0];
15            int barrelPort = Integer.parseInt(parts[1]);
16            String barrelHost = parts[2];
17
18            // Nao retransmitir para si mesmo
19            if (barrelName.equals(this.name) && barrelPort == this.port
&& barrelHost.equals(this.host)) {
20                continue;
21            }
22
23            try {
24                Registry barrelRegistry = LocateRegistry.getRegistry(
barrelHost, barrelPort);
25                BarrelInterface otherBarrel = (BarrelInterface)
barrelRegistry.lookup(barrelName);
26                otherBarrel.addToIndex(word, url, false); // Retransmite
```

```
27         } catch (Exception e) {
28             Utils.printLogException("Erro ao retransmitir para o
barrel " + barrelName + ": " + e.getMessage(), e);
29         }
30     }
31     } catch (Exception e) {
32         Utils.printLogException("Erro ao obter lista de barrels do
gateway: " + e.getMessage(), e);
33     }
34 }
35
36 public synchronized void addToIndex(String word, String url, boolean
shouldRetransmit) throws java.rmi.RemoteException {
37     String messageId = word + ":" + url; // ID unico da mensagem
38
39     // Verificar se ja recebeu esta mensagem
40     if (receivedMessages.contains(messageId)) {
41         return; // Duplicado, ignorar
42     }
43
44     receivedMessages.add(messageId); // Marcar como recebida
45
46     if (stopWordsManager.isStopword(word)) {
47         return; // Ignorar stopwords
48     }
49
50     if (indexedItems.containsKey(word)){
51         HashSet<String> urlsForWord = indexedItems.get(word);
52         urlsForWord.add(url);
53         indexedItems.put(word, urlsForWord);
54     } else {
55         HashSet<String> newUrlsForWord = new HashSet<String>();
56         newUrlsForWord.add(url);
57         indexedItems.put(word, newUrlsForWord);
58     }
59
60     // RETRANSMITIR de forma assincrona (fora do synchronized)
61     if (shouldRetransmit) {
62         retransmitExecutor.submit(() -> retransmitToOtherBarrels(word,
url));
63     }
64
65     // Notifica a gateway que index mudou
66     try {
67         ConfigReader gatewayConfig = new ConfigReader("gateway");
68         Registry gatewayRegistry = LocateRegistry.getRegistry(
        gatewayConfig.getHost(),
69         gatewayConfig.getPort()
70     );
71     };
72     GatewayInterface gateway = (GatewayInterface) gatewayRegistry.
lookup(gatewayConfig.getName());
73     gateway.notifyIndexChanged(name, port);
74     } catch (Exception ignore) {}
75 }
```

Listing 5: Retransmissão para replicação

3.3 Detecção Dinâmica de Stopwords

3.3.1 Algoritmo de Detecção

O sistema utiliza análise estatística para identificar stopwords dinamicamente:

1. **Contagem de frequências:** Para cada página, contar ocorrências de palavras
2. **Detecção de outliers:** Usar método IQR (Interquartile Range)

$$Q1 = \text{Percentil } 25$$

$$Q3 = \text{Percentil } 75$$

$$IQR = Q3 - Q1$$

$$\text{Threshold} = Q3 + (1.2 \times IQR)$$

3. **Análise cross-document:** Palavra torna-se stopwords se aparecer como outlier em $\geq 70\%$ dos documentos
4. **Linguagem:** Através da biblioteca *tika-core*, o downloader identifica a linguagem da página e as stopwords são detetadas e organizadas por língua. Caso não seja possível, identificar essas palavras ficam marcadas como *unknown*.
5. **Sincronização:** Quando stopwords mudam, retransmitir para todos os Barrels. Se um Barrel for inicializado, as stopwords existentes são sincronizadas.

```

1 private static final double STOPWORD_THRESHOLD = 0.40;
2 private static final double OUTLIER_K = 1.2;
3 private static final int MIN_WORD_FREQUENCY = 3;
4 private static final int MIN_DOCUMENTS_FOR_STOPWORDS = 2;

```

Listing 6: Configuração de stopwords

3.4 URL Queue

3.4.1 Gestão de Prioridades

A fila implementa duas prioridades:

- **Alta prioridade:** URLs adicionados manualmente pelo utilizador (frente da fila)
- **Baixa prioridade:** URLs descobertos pelo Downloader (fim da fila)

```

1 private BlockingDeque<String> urlsToIndex;
2 private Set<String> seenUrls;
3
4 public void putNew(String url, boolean priority) throws RemoteException
5 {
6     if (!seenUrls.add(url)) return;
7     if (priority)
8         urlsToIndex.putFirst(url);
9     else
10        urlsToIndex.putLast(url);
11 }

```

Listing 7: Implementação de fila com prioridade

3.5 Downloader

3.5.1 Pipeline de Processamento

1. Obter URL da fila (operação bloqueante)
2. Download da página com JSoup
3. Tokenização do conteúdo textual
4. Para cada palavra:
 - Converter para minúsculas
 - Remover caracteres não alfanuméricos
 - Filtrar palavras com < 3 caracteres
 - Contar frequência
 - Enviar para todos os Barrels ativos via `addToIndex(shouldRetransmit=true)`
5. Extrair todos os links `<a href>`
6. Adicionar links à fila (baixa prioridade)
7. Enviar backlinks para Barrels via `addUrlsForIndexedUrl()`
8. Enviar frequências para análise de stopwords via `addWordCounts()`

3.5.2 Retry Logic

O Downloader implementa retry com backoff:

- 3 tentativas por URL
- Timeout de 2 segundos por tentativa
- Em caso de falha, após as tentativas, o URL é descartado

3.6 Persistência e Recuperação

3.6.1 Backup de Estado

Os Barrels guardam estado em disco (`barrelX_progress.dat`):

- Índice invertido (`indexedItems`)
- Grafo de URLs (`urlsIndexed`)
- Backup da fila (`urlsToIndex`, `seenUrls`)

3.6.2 Recuperação após Falha

1. Ao iniciar, o Barrel verifica existência do ficheiro de progresso
2. Se existe: carrega esse ficheiro e sincroniza os dados em falta com os outros Barrels ativos - *syncBarrelWithOthers*
3. Se não: copia a informação dos Barrels ativos
4. Por fim, regista-se no Gateway

4 Backend - WebApp (Spring Boot)

4.1 Arquitetura em Camadas

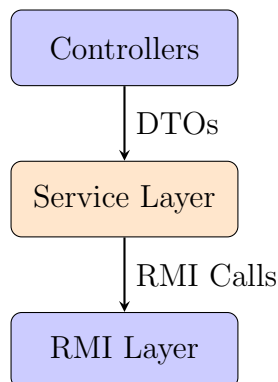


Figura 2: Arquitetura em camadas do WebApp

4.2 REST API Endpoints

Endpoint	Método	Descrição
/api/search?q={query}&page={page}	GET	Pesquisa simples com query params
/api/hackernews/index-top50	POST	Pesquisa avançada com JSON body
/api/connections?url={url}&page={page}	GET	Obter backlinks de um URL
/api/barrels/active	GET	Listar Barrels ativos
/api/barrels/registered	GET	Listar Barrels registrados
/api/barrels/add-url	POST	Adicionar URL ao índice
/api/statistics	GET	Obter estatísticas do sistema
/api/health	GET	Health check
/api/info	GET	Informação do sistema
/api/context-analysis	POST	Análise contextual com IA

Tabela 1: Endpoints da REST API

4.3 DTOs (Data Transfer Objects)

4.3.1 SearchResponseDTO

```

1 public class SearchResponseDTO {
2     private String query;
3     private List<SearchResultDTO> results;
4     private int currentPage;
5     private int pageSize;
6     private int totalResults;
7     private int totalPages;
8     private boolean hasResults;
9 }
  
```

Listing 8: Estrutura de resposta de pesquisa

4.3.2 SearchResultDTO

```
1 public class SearchResultDTO {
2     private String url;
3     private String title;
4     private String description;
5     private int references;
6 }
```

Listing 9: Estrutura de resultado individual

4.4 Segurança e SSL

O WebApp expõe HTTPS na porta 8443, usando um keystore **PKCS12** embebido no classpath, protegendo todas as rotas `/api/**` durante o desenvolvimento.

```
1 # Web Server info
2 server.port=8443
3 server.servlet.context-path=/
4
5 # SSL Configuration (Spring Boot)
6 server.ssl.enabled=true
7 server.ssl.key-store=classpath:keystore.p12
8 server.ssl.key-store-type=PKCS12
9 server.ssl.key-store-password=dev-ssl-password
10 server.ssl.key-password=dev-ssl-password
11 server.ssl.key-alias=tomcat
```

Listing 10: application.properties - SSL

4.5 Integração RMI via Spring

Cria um bean do tipo *GatewayInterface* conectando ao servidor RMI. Este bean é criado apenas uma vez na inicialização sendo reutilizado em todas as requisições. O spring boot injeta automaticamente onde for necessário via *@code* e *@Autowired*.

```
1 @Configuration
2 public class RMISConfig {
3     @Bean
4     public GatewayInterface gatewayService() throws Exception {
5         ...
6         Registry registry = LocateRegistry.getRegistry(gatewayHost,
7 gatewayPort);
8
9         GatewayInterface gateway = (GatewayInterface) registry.
10 lookup(gatewayName);
11
12         gateway.getActiveBarrels();
13
14         System.out.println("Conectado ao Gateway RMI com sucesso!");
15         return gateway;
16     }
17 }
```

Listing 11: RMISConfig.java - Bean de Gateway

4.6 Análise Contextual com IA

O sistema integra com a Google Gemini API para análise contextual (chamada direta por `URLConnection`; chave lida de variáveis de ambiente ou do ficheiro `Config.properties`):

```
1 @PostMapping("/context-analysis")
2 public ResponseEntity<Map<String, Object>> getContextAnalysis(
3     @RequestBody Map<String, String> request) {
4
5     String query = request.get("query");
6     String citations = request.get("citations");
7
8     String prompt = String.format(
9         "Analisa a seguinte pesquisa: '%s'. " +
10         "Contexto dos resultados: %s",
11         query, citations
12     );
13
14     // Chamada ao OpenRouter API
15     Map<String, Object> response = openRouterService.analyze(prompt);
16     return ResponseEntity.ok(response);
17 }
```

Listing 12: ContextAnalysisController.java

4.7 Hackernews

O sistema implementa integração automática com a API pública do HackerNews para indexação das 50 top storys. A integração consiste em dois componentes principais: `HackerNewsService` e `HackerNewsController`.

4.7.1 HackerNewsService

O serviço é responsável por comunicar com a API Firebase do HackerNews e coordenar a indexação. Utiliza `HttpClient` nativo do Java para realizar requisições HTTP assíncronas.

```
1 private static final String HN_TOP_STORIES_URL =
2     "https://hacker-news.firebaseio.com/v0/topstories.json";
3 private static final String HN_ITEM_PAGE_URL =
4     "https://news.ycombinator.com/item?id=%d";
5 private static final int TOP_STORIES_LIMIT = 50;
```

Listing 13: Endpoints da API HackerNews

4.7.2 Fluxo de Indexação

1. Ao iniciar a aplicação Spring Boot, o `@EventListener(ApplicationReadyEvent.class)` dispara automaticamente.
2. Aguarda 5 segundos para garantir que Gateway e Barrels estão ativos
3. Faz GET em `topstories.json` para obter array de IDs

4. Limita aos primeiros 50 IDs
5. Para cada ID, constrói URL: `news.ycombinator.com/item?id={id}`
6. Invoca `gatewayClient.addURL(url, false)` via RMI
7. Aplica rate limiting de 100ms entre requisições
8. URLs entram na fila com alta prioridade
9. Downloaders processam normalmente (download, parsing, indexação)

4.7.3 Controlo de Execução

A indexação automática pode ser desativada no arranque do Googol via `application.properties`:

```
1 hackernews.index-on-startup=true
```

Listing 14: Propriedade de controlo

Caso o utilizador queira posteriormente indexar e tenha arrancado com a flag a false, terá de ir à página /indexar, usando o botão dedicado.

4.7.4 Identificação de Conteúdo HackerNews

Os resultados provenientes do HackerNews são identificados no frontend através da verificação de domínio:

```
1 private boolean isHackerNewsUrl(String url) {  
2     return url != null && url.contains("news.ycombinator.com");  
3 }  
4  
5 return new SearchResultDTO(  
6     url,  
7     pageInfo.getTitle(),  
8     pageInfo.getDescription(),  
9     references,  
10    isHackerNewsUrl(url)  
11 );
```

Listing 15: Marcação de URLs do HackerNews

No frontend, o componente `AnimatedList` exibe um badge distintivo para itens do HackerNews, permitindo aos utilizadores identificar visualmente conteúdo proveniente desta fonte.

4.7.5 Vantagens da Integração

- **Conteúdo de qualidade:** HackerNews é curado pela comunidade
- **Diversidade:** Tópicos de tecnologia, startups, ciência
- **Atualização automática:** Top 50 muda diariamente
- **Grafo rico:** Páginas linkadas expandem índice organicamente
- **Demonstração:** Indexação rápida para testes e demos

5 Frontend - React/Next.js

5.1 Stack Tecnológica

- **Framework:** Next.js 16 (React 19) - App Router
- **Linguagem:** TypeScript (strict mode)
- **Animações:** Framer Motion, GSAP
- **WebGL:** OGL (shader 3D para orbe animado)
- **UI:** HeroUI, componentes custom

5.2 Estrutura de Rotas

Rota	Descrição
/	Página inicial com interface de pesquisa
/results	Resultados de pesquisa paginados
/indexar	Formulário para adicionar URLs
/ligacoes	Formulário para pesquisar backlinks
/ligacoes/results	Resultados de backlinks
/statistics	Dashboard de estatísticas em tempo real
/autores	Página da equipa

Tabela 2: Rotas do frontend

5.3 Componentes Principais

5.3.1 AnimatedList

Lista animada de resultados com funcionalidades:

- Navegação por teclado (setas, Tab, Enter)
- Expansão/colapso de detalhes
- Auto-scroll para item selecionado
- Gradientes de fade no topo e base
- Animações com Framer Motion

5.3.2 Orb (WebGL)

Orbe 3D animado com shaders GLSL:

- Simplex noise para distorção
- Rotação em hover

- Mudança de cor por hue (0-360°)
- Efeito wave baseado na posição do rato
- 60 FPS com requestAnimationFrame

```
1 vec3 color1 = hsv2rgb(vec3(hue / 360.0, 0.8, 1.0));  
2 vec3 color2 = hsv2rgb(vec3((hue + 60.0) / 360.0, 0.9, 0.8));  
3  
4 float noise = snoise(vPosition * noiseScale + uTime * 0.2);  
5 vec3 finalColor = mix(color1, color2, noise * 0.5 + 0.5);
```

Listing 16: Shader GLSL do Orb (fragmento)

5.3.3 Cursor Customizado

Cursor com trail de 30 círculos:

- Física de spring (30% damping)
- Cores adaptativas por rota
- Z-index alto (99999999)
- Esconde cursor padrão

5.3.4 StaggeredMenu

Menu lateral com animações GSAP:

- 3 camadas de background com entrada staggered
- Itens com rotação + fade-in
- Efeito de texto alternado (Menu ↔ Close)
- Rotação de ícone (0° → 225°)

5.3.5 ProfileCard

Cards 3D com efeito holográfico:

- Tilt baseado na posição do rato
- Perspetiva 3D com CSS transforms
- Suporte a orientação do dispositivo (mobile)
- Links para redes sociais
- Efeito de shine holográfico

5.4 Sistema de Cores por Rota

Rota	Cor Primária	Cor Secundária	Hue
Home/Results	#9c43ff (Roxo)	#4cb8e9 (Azul)	0°
Indexar	#ff6b9d (Rosa)	#ff8c42 (Laranja)	270°
Ligações	#00ff88 (Verde)	#88ff00 (Amarelo)	117°
Estatísticas	#ff3333 (Vermelho)	#cc0000 (Verm. Escuro)	0°

Tabela 3: Sistema de cores adaptativo

5.5 Funcionalidades Avançadas

5.5.1 Análise Contextual com IA

- Botão info flutuante ao lado da barra de pesquisa
- Hover para mostrar análise
- Cache de 30 minutos por query
- Envia query + top 5 descrições para backend
- Exibe insights contextuais

5.5.2 Estatísticas em Tempo Real

- SSE via EventSource (sem polling)
- Eventos: “stats”, “barrels-active”, “barrels-registered”
- Snapshot inicial via REST e reconciliação por eventos
- Tempo médio de resposta e Top 10 pesquisas
- Lista unificada de Barrels (ativos/registados)

5.5.3 Paginação Inteligente

- Mostra primeiras 3, últimas 3, e atual ± 2 páginas
- Ellipsis para páginas omitidas
- Sincronização com URL (?page=N)
- Scroll suave ao mudar de página

5.6 Otimizações de Performance

1. **Code splitting:** Automático via Next.js
2. **Animações:** `requestAnimationFrame` para 60 FPS
3. **Cache de API:** Análise contextual em cache (30 min)
4. **WebGL:** Contexto único, `cleanup` em `unmount`
5. **Memory management:** `Cleanup` em todos os `useEffect`

5.7 Acessibilidade

- HTML semântico (`<main>`, `<nav>`, `<section>`)
- Labels ARIA em todos os botões de ícone
- Navegação completa por teclado
- Estados de focus visíveis
- Alto contraste (branco em fundo escuro)
- Suporte a screen readers

6 Comunicação Entre Componentes

6.1 RMI (Remote Method Invocation)

6.1.1 Configuração de Registry

```
1 gateway.host=127.0.0.1
2 gateway.port=8183
3 gateway.name=gateway
4
5 queue.host=127.0.0.1
6 queue.port=8181
7 queue.name=urlqueue
8
9 barrel1.host=127.0.0.1
10 barrel1.port=8182
11 barrel1.name=barrel1
12
13 barrel2.host=127.0.0.1
14 barrel2.port=8184
15 barrel2.name=barrel2
16
17 barrel3.host=127.0.0.1
18 barrel3.port=8185
19 barrel3.name=barrel3
```

Listing 17: Configuração RMI (Config.properties)

6.1.2 Padrão de Callbacks

O sistema implementa 2 callbacks (Estatísticas e Estado dos Barrels) para notificações em tempo real:

```
1 public interface StatsCallback extends Remote {
2     void onStatsUpdate(Map<String, Object> stats) throws RemoteException
3     ;
4 }
```

Listing 18: StatsCallback interface

Uso:

1. Cliente registra callback no Gateway
2. Gateway armazena callback em `CopyOnWriteArrayList`
3. Em mudança de estado, Gateway chama `callback.onStatsUpdate()`
4. Cliente recebe e exibe estatísticas atualizadas
5. Callbacks mortos removidos automaticamente em erro

6.2 REST API (HTTPS/JSON)

6.2.1 Exemplo de Request/Response

Request:

```
1 GET /api/search?q=distributed%20systems&page=0&pageSize=10
```

Response:

```
1 {
2   "query": "distributed systems",
3   "results": [
4     {
5       "url": "https://example.com",
6       "title": "Introduction to Distributed Systems",
7       "description": "Learn about distributed computing...",
8       "references": 42
9     }
10  ],
11  "currentPage": 0,
12  "pageSize": 10,
13  "totalResults": 5,
14  "totalPages": 1,
15  "hasResults": true
16 }
```

6.3 CORS (Cross-Origin Resource Sharing)

```
1 @Configuration
2 public class CorsConfig {
3     @Bean
4     public CorsFilter corsFilter() {
5         CorsConfiguration config = new CorsConfiguration();
6         config.setAllowCredentials(true);
7         // HTTP
8         config.addAllowedOrigin("http://localhost:3000");
9         config.addAllowedOrigin("http://localhost:3001");
10        config.addAllowedOrigin("http://localhost:3002");
11        // HTTPS
12        config.addAllowedOrigin("https://localhost:3000");
13        config.addAllowedOrigin("https://localhost:3001");
14        config.addAllowedOrigin("https://localhost:3002");
15        // Headers/Methods
16        config.addAllowedHeader("*");
17        config.addAllowedMethod("*");
18
19        UrlBasedCorsConfigurationSource source =
20            new UrlBasedCorsConfigurationSource();
21        source.registerCorsConfiguration("/api/**", config);
22
23        return new CorsFilter(source);
24    }
25 }
```

Listing 19: Configuração CORS

7 Estatísticas e Monitorização

7.1 Métricas Coletadas

1. Estatísticas de Pesquisa:

- $\text{Map}\langle \text{String}, \text{Integer} \rangle$ - Palavra $\rightarrow$ Contagem
- Top 10 pesquisas mais frequentes

2. Tempo de Resposta:

- $\text{Map}\langle \text{String}, \text{Long} \rangle$ - Barrel $\rightarrow$ Tempo total (ns)
- $\text{Map}\langle \text{String}, \text{Long} \rangle$ - Barrel $\rightarrow$ Contagem de requests
- Média = Tempo total / Contagem

3. Estado de Barrels:

- Lista de Barrels ativos
- Lista de Barrels registados
- Tamanho de índice por Barrel

7.2 Dashboard de Estatísticas

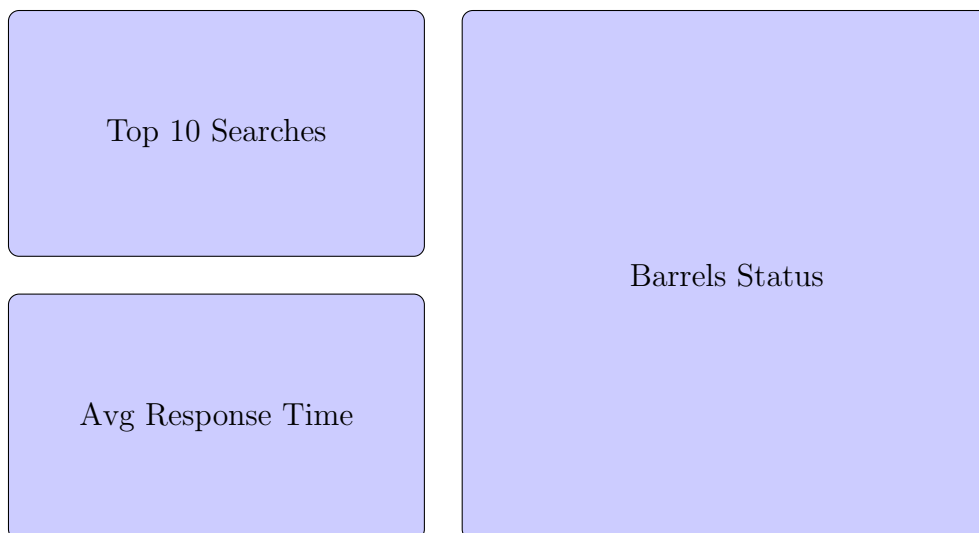


Figura 3: Layout do dashboard de estatísticas

7.3 Atualização em Tempo Real

7.3.1 SSE no Frontend

```
1 // Subscribe at mount, cleanup at unmount
2 useEffect(() => {
3   const unsubscribe = subscribeStatistics(
```

```
4      (stats) => setStats(stats),
5      (active) => setActiveBarrels(active),
6      (registered) => setRegisteredBarrels(registered)
7  );
8  return () => unsubscribe();
9 }, []);
```

Listing 20: Assinatura SSE com EventSource

7.3.2 Callbacks RMI (Backend)

```
1 private void notifyStatsUpdateAsync() {
2     for (StatsCallback cb : statsCallbacks) {
3         callbackExecutor.submit(() -> {
4             try {cb.onStatsUpdate(stats.getStatistics());}
5             catch (Exception e) {
6                 statsCallbacks.remove(cb);
7             }
8         });
9     }
10 }
11
12 private void notifyActiveBarrelsUpdateAsync() {
13     List<String> snapshot;
14     try {snapshot = getActiveBarrels();}
15     catch (RemoteException e) {
16         snapshot = new ArrayList<>();
17     }
18     final List<String> msg = snapshot;
19     for (BarrelStateCallback cb : barrelStateCallbacks) {
20         callbackExecutor.submit(() -> {
21             try {cb.onActiveBarrelsUpdate(msg);}
22             catch (Exception e) {
23                 barrelStateCallbacks.remove(cb);
24             }
25         });
26     }
27 }
28
29 private void notifyRegisteredBarrelsUpdateAsync() {
30     List<String> snapshot;
31     try {snapshot = getRegisteredBarrels();}
32     catch (RemoteException e) {
33         snapshot = new ArrayList<>();
34     }
35     final List<String> msg = snapshot;
36     for (BarrelStateCallback cb : barrelStateCallbacks) {
37         callbackExecutor.submit(() -> {
38             try {cb.onRegisteredBarrelsUpdate(msg);}
39             catch (Exception e) {barrelStateCallbacks.remove(cb);}
40         });
41     }
42 }
```

Listing 21: Notificação assíncrona

8 Páginas da Aplicação

8.1 Home (/)

A página inicial fornece a entrada para o fluxo de pesquisa. O componente **SearchBar** valida a query, persiste o termo na *query string* e redireciona para `/results?page=0`. Elementos visuais como **Header**, **Orb** (WebGL) e **StaggeredMenu** compõem a identidade visual sem impactar o caminho crítico de interação.

8.2 Resultados (/results)

Carrega resultados via `GET /api/search?q=...&page=N`. O algoritmo de paginação é estável e sincronizado com a URL, mantendo scroll suave e preservando o estado entre navegações. Os itens são exibidos por **AnimatedList** (navegação por teclado e foco visível). A análise contextual (**AIDropdown**) agrega um sumário da query alimentado por `/api/context-analysis`, com cache leve em memória por query para evitar recomputações e chamadas redundantes. Cada entrada mostra **title**, **description** (derivado de **PageInfo**) e **references** (soma de inbound links agregada no Gateway).

8.3 Ligações (/ligacoes e /ligacoes/results)

A página de entrada valida o URL (formato canónico, protocolo obrigatório) e redireciona para `/ligacoes/results?url=...&page=0`. A página de resultados invoca `GET /api/connections?url=...&page=N`, com paginação local consistente com o backend. É aplicado o mesmo esqueleto de **AnimatedList** para coerência de UX.

8.4 Indexar (/indexar)

Submete `POST /api/barrels/add-url` com `url` e, em caso de URL previamente visto, o backend devolve `alreadyIndexed=true`. O UI apresenta um modal de confirmação para *reindex* com `indexAnyway=true`. Inclui ainda a indexação opcional de HackerNews via `POST /api/hackernews/index-top50`, refletindo um conector externo simples.

8.5 Estatísticas (/statistics)

Faz *bootstrap* do estado com `GET /api/statistics` e inicia a subscrição SSE em `/api/statistics/stream`. Os eventos nomeados `"stats"`, `"barrels-active"` e `"barrels-registered"` atualizam o estado reativo. O layout apresenta Top 10 pesquisas, tempos médios por Barrel (ms) e uma lista unificada de Barrels com extração `name:port:host:indexSize` e estado (ativo/registado), coerente com os snapshots fornecidos pelo Gateway.

8.6 Autores (/autores)

Página estática responsiva com **ProfileCard** e efeitos gráficos, servindo como identificação da equipa e ponte para contactos.

9 Integrações

9.1 Spring Boot + RPC/RMI

A camada WebApp (Spring Boot) atua como *BFF* (Backend for Frontend) exposta em HTTPS (porta 8443), traduzindo requisições REST para invocações RMI no `GatewayInterface`. A composição de respostas aplica DTOs, enriquece resultados com `PageInfo` e isola falhas RMI com tratamento de exceções e *timeouts*.

9.2 REST WebServices

A API pública expõe endpoints REST JSON sob `/api`, com CORS configurado para origens `localhost:3000-3002` em HTTP e HTTPS. SSL encontra-se ativo por `keystore.p12` (PKCS12) no classpath, garantindo confidencialidade ponto-a-ponto em desenvolvimento.

9.3 SSE e plano de WebSockets

As atualizações em tempo real adotam SSE (`SseEmitter`) por simplicidade e alinhamento com o padrão de emissor único (servidor→cliente). Caso seja requerido canal bidirecional (notificações cliente→servidor), o plano de evolução consiste em habilitar WebSockets (STOMP/SockJS) no Spring e publicar os mesmos tópicos `"stats"`, `"barrels-active"`, `"barrels-registered"`, mantendo a mesma semântica de mensagens.

9.4 Integração opcional com FastAPI

Como alternativa/*gateway* paralelo, uma aplicação FastAPI pode expor endpoints equivalentes e consumir o WebApp via REST ou comunicar com o ecossistema Java por uma ponte (p.ex., gRPC/HTTP para um adaptador Spring). O desenho preserva RMI para tráfego intra-JVM e usa FastAPI apenas como *edge* HTTP para casos específicos (p.ex., *prototyping* de features de AI), mantendo coerência de contratos JSON.

10 Testes de Software

10.1 Tabela de testes e resultados

ID	Caso	Descrição	Resultado
T1	GET /api/health	API UP, Gateway ligado/desligado refletido no JSON	Passou
T2	GET /api/search	<code>totalResults</code> coerente com <code>results</code> e paginação	Passou
T3	POST /api/search	Corpo JSON com múltiplas palavras, interseção correta	Passou
T4	POST /api/barrels/add-url	URL novo retorna <code>success=true</code>	Passou
T5	Reindex	URL já indexado retorna <code>alreadyIndexed</code> e aceita <code>indexAnyway=true</code>	Passou
T6	GET /api/connections	Backlinks paginados, <code>totalPages</code> consistente	Passou
T7	GET /api/statistics	Snapshot com Top 10 e médias por Barrel	Passou
T8	SSE /api/statistics/stream	Eventos <code>stats</code> , <code>barrels-active</code> , <code>barrels-registered</code>	Passou
T9	CORS	Origens <code>http/https</code> <code>localhost:3000-3002</code> permitidas	Passou
T10	SSL	Handshake válido com <code>https://localhost:8443</code>	Passou
T11	Failover RMI	Queda de um Barrel não interrompe pesquisa (retry/round-robin)	Passou
T12	/api/context-analysis	Responde com análise quando chave Gemini ativa; erro tratável sem chave	Passou
T13	HackerNews	<code>/hackernews/index-top50</code> indexa itens e responde com contagem	Passou

Tabela 4: Casos de teste funcionais com resultado observado

10.2 Cobertura de cenários

Os testes cobrem o *happy path* e condições de falha expectáveis (CORS, ausência de chave de IA, desconexão de Barrels). A validação de consistência de paginação e de interseção multi-palavra assegura a correção do algoritmo de ranking por referências agregadas.

11 Distribuição de Tarefas

Henrique Diz — Liderança de frontend: arquitetura Next.js, componentes avançados (`AnimatedList`, `Orb`, `StaggeredMenu`), acessibilidade e integração SSE com `/statistics/stream`. Responsável pela página de Resultados e *UX* geral.

Rodrigo Manão — Núcleo backend distribuído: `Gateway`, `IndexStorageBarrel`, `URLQueue` e `Downloader`. Implementação de replicação, stopwords dinâmicas (IQR), failover e estatísticas. Hardening de serialização e recuperação de estado.

João Francisco — WebApp Spring Boot: controllers REST, DTOs, `GatewayServiceClient`, CORS/SSL, SSE (`SseEmitter`) e integração de IA (Google Gemini). Automação com scripts e endpoints de suporte (`health/info/HackerNews`).

A colaboração incluiu revisões cruzadas, testes integrados ponta-a-ponta e documentação técnica deste relatório.

12 Conclusão

12.1 Objetivos Alcançados

O projeto Googol implementou com sucesso:

- **Arquitetura distribuída** com múltiplos componentes RMI
- **Balanceamento de carga** round-robin entre Barrels
- **Tolerância a falhas** com retry automático e replicação
- **Índice invertido** distribuído e replicado
- **Deteção dinâmica de stopwords** com análise estatística
- **Ranking por PageRank** baseado em inbound links
- **API REST** moderna com Spring Boot
- **Frontend responsivo** com animações avançadas e WebGL
- **Análise contextual com IA** via Google Gemini
- **Estatísticas em tempo real** com dashboard interativo

12.2 Destaques Técnicos

12.2.1 Backend

- Comunicação RMI robusta com failover
- Replicação assíncrona de índice
- Persistência com serialização Java
- Callbacks para notificações em tempo real
- Integração Spring Boot + RMI

12.2.2 Frontend

- WebGL shaders customizados (orbe 3D)
- Animações GSAP com timelines complexas
- Cursor customizado com física de spring
- Cards holográficos 3D
- Sistema de cores adaptativo por rota
- Paginação inteligente com ellipsis

12.3 Pontos Fortes

1. **Escalabilidade:** Arquitetura permite adicionar Barrels dinamicamente
2. **Resiliência:** Sistema continua operacional com falha de componentes
3. **Performance:** Índice distribuído permite pesquisas paralelas
4. **UX excepcional:** Interface moderna com animações fluidas
5. **Inovação:** Detecção automática de stopwords e análise com IA

12.4 Possíveis Melhorias Futuras

1. **Sharding:** Distribuir índice por hash de palavra (não replicação total)
2. **Caching:** Adicionar camada de cache Redis para queries frequentes
3. **Ranking:** Implementar TF-IDF e algoritmo completo de PageRank
4. **Crawling:** Downloader com rate limiting e robots.txt
5. **Frontend:** Server-side rendering para SEO
6. **Segurança:** Autenticação e rate limiting na API
7. **Monitorização:** Prometheus + Grafana para métricas

12.5 Considerações Finais

O Googol demonstra uma implementação completa e funcional de um motor de busca distribuído, combinando conceitos avançados de sistemas distribuídos (RMI, replicação, balanceamento) com tecnologias web modernas (Spring Boot, React, WebGL). O sistema é robusto, escalável e fornece uma excelente experiência ao utilizador.

A arquitetura modular permite fácil extensão e manutenção, enquanto os mecanismos de failover e replicação garantem alta disponibilidade. A interface moderna com animações WebGL e análise contextual com IA elevam o projeto além de um simples motor de busca académico.

A Configuração do Ambiente

A.1 Requisitos

- Java 21
- Maven 3.8+
- Node.js 18+
- npm 9+

A.2 Instalação

A.2.1 Backend

```
1 cd backend
2 mvn clean install
```

A.2.2 Frontend

```
1 cd frontend
2 npm install
```

A.3 Execução

A.3.1 Componentes RMI

```
1 # URL Queue
2 mvn exec:java -Dexec.mainClass="queue.URLQueue"
3
4 # Gateway
5 cd backend
6 mvn exec:java -Dexec.mainClass="gateway.Gateway"
7
8 # Barrel 1
9 mvn exec:java -Dexec.mainClass="barrel.IndexStorageBarrel" \
10   -Dexec.args="barrel1"
11
12 # Barrel 2
13 mvn exec:java -Dexec.mainClass="barrel.IndexStorageBarrel" \
14   -Dexec.args="barrel2"
15
16 # Downloader
17 mvn exec:java -Dexec.mainClass="downloader.Downloader"
```

A.3.2 Frontend

```
1 cd backend
2 make run-frontend
```

B Estrutura de Ficheiros

B.1 Backend

```
1 backend/  
2     src/main/java/  
3         barrel/  
4         client/  
5         common/  
6         downloader/  
7         gateway/  
8         queue/  
9         statistics/  
10        webapp/  
11            controller/  
12            service/  
13            dto/  
14            config/  
15        src/main/resources/  
16            application.properties  
17            Config.properties  
18            keystore.p12  
19        files/  
20            Log_Exceptions.txt  
21        barrel1_progress.dat  
22        barrel2_progress.dat  
23        makefile  
24        pom.xml
```

B.2 Frontend

```
1 frontend/  
2     certificates/  
3         localhost-key.pem  
4         localhost.pem  
5     certs/  
6         cert.pem  
7         key.pem  
8     public/  
9         Henrique.jpg  
10        Joao.jpg  
11        Rodrigo.jpg  
12     src/  
13         app/  
14             autores/page.tsx  
15             indexar/page.tsx  
16             ligacoes/  
17                 page.tsx  
18                 results/page.tsx  
19             results/page.tsx  
20             statistics/page.tsx  
21             layout.tsx  
22             page.tsx  
23         components/
```

```
24         AIDropdown/  
25         AnimatedList/  
26         ApiStatus/  
27         Cursor/  
28         GradientText/  
29         Header/  
30         Loader/  
31         Modal/  
32         Noise/  
33         Orb/  
34         ProfileCard/  
35         SearchBar/  
36         StaggeredMenu/  
37     services/  
38     api.ts  
39     .env.local  
40     next.config.js  
41     package.json  
42     package-lock.json  
43     server.js  
44     tsconfig.json  
45     .gitignore
```